

Blinkit Demand Forecast Engine

The Cradle Box

May 2026

CONTENTS

Blinkit Demand Forecast Engine	
How We Project Future Sales, Month by Month	
The One Question We're Answering	
Forecast vs Replenishment — The Key Distinction	
Two Models, One System	
The Blinkit Forecast Formula	
Step 1 — Compute mean_all ADS per SKU	
raw_ADS (per DS)	
mean_all (per SKU)	
Percentile signals also computed	
Step 2 — Count Dark Stores for Each Future Month	
Two Components of DS Count	
Step 3 — Compute the Forecast	
Fallback Hierarchy	
Step 4 — Store as VELOCITY_BASE	
Step 5 — Manual Override (USER_FINAL)	
The Full Flow, End to End	
A Worked Example	
Known Limitations	

How to Read the Forecast Tab

Key Numbers

Blinkit Demand Forecast Engine

How We Project Future Sales, Month by Month

The One Question We're Answering

“How many units of each SKU will sell on Blinkit in each of the next 6 months?”

This is a forward-looking projection used for production planning, purchase order sizing, and channel strategy. It is **not** the same question as replenishment (“what do I ship this week?”). The forecast looks months ahead; replenishment looks at current stock levels.

Forecast vs Replenishment — The Key Distinction

	Replenishment Engine	Forecast Engine
Question	What to ship to WH this week	How many units sell in month X
Horizon	0–37 days	1–6 months forward
Output	Units to ship per WH × SKU	Units sold per SKU × month
DS count	Actual active DSes today	Current DSes + planned city expansion
ADS signal	Imputed ADS (corrects for OOS)	mean_all (includes zero-ADS DSes)
OOS treatment	Fill OOS days with P75 estimate	Exclude from ADS; use penetration signal
Granularity	WH level	Channel level (all Blinkit consolidated)

Two Models, One System

The forecast engine runs two parallel models and merges their outputs:

Model 1 — Blinkit ADS Model (this document) For Blinkit only. Uses daily sales velocity data and a city-by-city expansion plan.

Model 2 — Historical Growth Model For Amazon, FnP, First Cry, Peeko, Ozi. Uses month-over-month growth rates from order history. Not covered here.

D2C is manual only — no engine forecast; you enter D2C numbers by hand.

The Blinkit Forecast Formula

For each SKU, for each future month:

$$\text{forecast_units} = \text{p_central} \times \text{DS_count} \times 30$$

Where:

- **p_central** = mean_all ADS — the expected daily orders per dark store
- **DS_count** = number of dark stores expected to carry the SKU in that month
- **30** = days in a month (fixed)

Three outputs per SKU × month:

Output	Formula	Meaning
<code>units</code> (central)	<code>p_central × DS_count × 30</code>	Expected outcome
<code>lo</code> (downside)	<code>p_central × 0.5 × DS_count × 30</code>	50% of central — pessimistic
<code>hi</code> (upside)	<code>p_90 × DS_count × 30</code>	P90 DS performance — optimistic

Step 1 — Compute mean_all ADS per SKU

The forecast uses **mean_all**, not P75. Here is why the two differ:

RAW_ADS (PER DS)

For each DS × SKU pair, over the most recent 30 days:

$$\text{raw_ADS} = \frac{\text{orders on available days}}{\text{count of available days}}$$

Available day = Column Q in performance CSV = `inventory_available = True`.

A DS must have **at least 5 available days** to be counted as reliable.

MEAN_ALL (PER SKU)

```
mean_all = AVERAGE of raw_ADS across ALL reliable DSes,  
           including zero-ADS DSes (in stock, no sale)
```

Why include zero-ADS DSes?

When we launch in a new city, we don't know which specific dark stores will sell and which won't. `mean_all` gives us the *expected value per new dark store* — it captures both the selling DSes (high ADS) and the non-selling DSes (ADS = 0) in a single realistic signal.

Example: if 4 out of 10 DSes sell 0.5 orders/day and 6 DSes sell 0:

```
mean_all = (4 × 0.5 + 6 × 0) / 10 = 0.20 orders/DS/day
```

P75 (used in replenishment) would give the 75th percentile DS performance — that's the top 25% performer. It's the right signal for "what would a typical active DS have sold on an OOS day?" but too optimistic for "what will an average new DS sell?"

PERCENTILE SIGNALS ALSO COMPUTED

Signal	Definition	Used for
<code>sku_p25_ads</code>	P25 of raw_ADS across reliable DSes	<code>lo</code> bound fallback
<code>sku_p75_ads</code>	P75 of raw_ADS across reliable DSes	Internal reference
<code>sku_p90_ads</code>	P90 of raw_ADS across reliable DSes	<code>hi</code> bound

Step 2 — Count Dark Stores for Each Future Month

This is where the forecast does something the replenishment engine cannot: it **looks forward** using an expansion plan.

TWO COMPONENTS OF DS COUNT

$$\text{DS_count} = \text{floor_DS} + \text{new_DS}$$

Floor DSes — Cities Already Live

For cities where the SKU is currently active:

$$\text{floor_DS} = \text{COUNT of DSes with status = 'active' in blinkit_ds_sku_eligibility}$$

This is a hard number from the database. It's what we have today.

New DSes — Cities in the Expansion Plan

You maintain a file: `data/blinkit/manual/City Launch Plan_Blinkit.xlsx`

This Excel has one row per SKU, with columns for each future month (Jun 2026, Jul 2026, ...). Each cell names the cities planned for that month. You can write "All Cities" to mean every city in our Blinkit footprint.

For new cities:

$$\begin{aligned} \text{raw_new_DS} &= \text{COUNT of all non-closed DSes in the planned city} \\ &\quad (\text{from blinkit_ds_sku_eligibility, all statuses except darkstore_closed}) \\ \text{new_DS} &= \text{round}(\text{raw_new_DS} \times (1 - \text{churn_rate})) \end{aligned}$$

Churn Rate — The Realism Correction

Not every dark store that lists your SKU keeps it. Blinkit redistributes slow movers. The churn rate measures how often that happens:

$$\text{churn_rate} = \frac{\text{sku_moved_out_low_sales DSes}}{(\text{active DSes} + \text{sku_moved_out_low_sales DSes})}$$

A churn rate of 20% means: for every 10 new dark stores you launch into, historically 2 of them lose your SKU within the assessment window. We apply this haircut to new city DS counts.

Step 3 — Compute the Forecast

With `p_central` (mean_all) and `DS_count` (floor + new):

```
# Central estimate
units = max(0, round( p_central x DS_count x 30 ))

# Downside (half of central)
lo     = max(0, round( p_central x 0.5 x DS_count x 30 ))

# Upside (P90 DS performance)
p_hi   = sku_p90_ads if sku_p90_ads > 0 else p_central x 1.3
hi     = max(0, round( p_hi x DS_count x 30 ))
```

FALLBACK HIERARCHY

Situation	What We Use
SKU has no data at all	Global median of <code>mean_all</code> across all SKUs
City P75 missing (new city)	Global <code>sku_p75</code> (all cities combined)
P90 is zero	<code>p_central x 1.3</code> as hi bound

Step 4 — Store as VELOCITY_BASE

All engine-generated rows are stored in the `demand_forecasts` table with `model = 'VELOCITY_BASE'`.

```
demand_forecasts
├ sku_id
├ channel_id (4 = Blinkit)
├ forecast_month (ISO date of first day of month)
├ forecast_units
├ confidence_lo
├ confidence_hi
└ model ('VELOCITY_BASE' or 'USER_FINAL')
```

`VELOCITY_BASE` rows are **overwritten** every time you click “Regenerate.” Your manual overrides are safe because they use `model = 'USER_FINAL'` — a separate conflict key.

Step 5 — Manual Override (USER_FINAL)

You can override any SKU × month in the Forecast tab of the dashboard.

How it works:

1. You edit a cell in the forecast grid and click “Lock Edited Cells”
2. The system writes `USER_FINAL` rows for that SKU × month
3. When displaying the forecast, `USER_FINAL` takes precedence over `VELOCITY_BASE`
4. When downloading the forecast Excel, locked cells are marked

Channel distribution: When you lock a total (e.g., “600 units in July”), the system splits it across channels proportionally to the `VELOCITY_BASE` channel mix. If Blinkit is 70% of engine output for that SKU, it gets 70% of your locked total.

Resetting: Click “Reset to Engine Forecast” to delete `USER_FINAL` rows for that cell. The `VELOCITY_BASE` shows through again.

The Full Flow, End to End

```

blinkit_performance_detail (DB)
  |
  v
fetch_blinkit_daily_ads()
├─ Load latest 30 days of DS-level data
└─ Paginate all (location_id, sku_id) pairs
  |
  v
compute_ds_ads()
├─ raw_ADS per DS = orders on avail days / avail days
├─ Filter: avail_days >= 5 to be "reliable"
├─ Unreliable DSes imputed with SKU median ADS
└─ Compute P25, P75, P90 across ALL reliable DSes (incl. zeros)
  |
  v
mean_all = average ADS across ALL reliable DSes per SKU
p90      = P90 of raw_ADS per SKU (for hi bound)
  |
  v
parse_city_launch_plan()
├─ Read City Launch Plan_Blinkit.xlsx
├─ Map future months → cities
└─ Normalise city names (Delhi = New Delhi, Bangalore = Bengaluru, etc.)
  |
  v
blinkit_ds_sku_eligibility (DB)
├─ active_cities_per_sku ← floor (what's live today)
├─ active_DS_per_sku_city ← DS count per city (for floor)
└─ churn_rate             ← sku_moved_out / (active + moved_out)
  |
  v
For each SKU × forecast month:
├─ floor_DS = active DSes in current active cities
├─ new_DS   = (planned new city DSes) × (1 - churn_rate)
├─ DS_count = floor_DS + new_DS
└─ units    = mean_all × DS_count × 30
  |
  v
demand_forecasts (DB)
└─ Upsert VELOCITY_BASE rows
    (USER_FINAL rows untouched)
  |
  v
Forecast tab – editable grid
├─ VELOCITY_BASE shown as base
└─ USER_FINAL overrides shown as locked cells

```

- └ Channel breakdown per SKU on drill-down
- └ Download forecast Excel (3 sheets)

A Worked Example

SKU: TCB008 (Just Arrived Bunny) **Forecast month:** July 2026

Current state (floor):

- Active in Bengaluru: 22 DSes
- Active in Mumbai: 16 DSes
- Active in Hyderabad: 6 DSes
- **Floor DS count = 44**

Expansion plan says: Launch in Chennai (18 DSes) in July

```
churn_rate for TCB008 = 2 / (22 + 2) = 8.3%
new_DS = 18 × (1 - 0.083) = 16.5 → 17
DS_count = 44 + 17 = 61
```

ADS signals (from performance data):

```
mean_all = 0.074 orders/DS/day
p90      = 0.333 orders/DS/day
```

Forecast:

```
units (central) = 0.074 × 61 × 30 = 135
lo              = 0.074 × 0.5 × 61 × 30 = 68
hi              = 0.333 × 61 × 30 = 610
```

Known Limitations

Limitation	Effect	Mitigation
City launch plan is manual	Forecast doesn't know about un-planned city launches	Update Excel before running forecast
WH-OOS blind spot	If WH was dry all period, mean_all = 0 for that WH → under-forecast	Use manual override (USER_FINAL) for known-good SKUs
New SKU, no history	No reliable DSes → global fallback ADS used	Forecast will be low; use USER_FINAL based on comparable SKU
TCB009 alias	TCB009 performance data stored as TCB009_1 in some files	Alias mapping handled in code; verify if ADS looks wrong
Inactive SKUs	TCB007, TCB010 excluded from Blinkit forecast by design	Hard-coded exclusion list in <code>BLK_EXCLUDE_SKUS</code>

How to Read the Forecast Tab

The forecast grid shows one row per SKU, one column per future month.

- **White cell** — engine-generated (`VELOCITY_BASE`). Recalculates on next regenerate.
- **Locked cell** — manual override (`USER_FINAL`). Persists across regenerates.
- **Base forecast row** — always shows the raw engine output, regardless of overrides.
- **Channel breakdown** — click any SKU to see how units split across Blinkit, Amazon, FnP etc.

When to lock cells:

- You have confirmed stock arriving that changes sellable supply
- You know a city launch is happening sooner/later than the plan shows
- The engine is producing an obviously wrong number (e.g. WH-OOS blind spot)

When to let the engine run free:

- Normal months with no special events
- Newly launched cities where you want to learn the real velocity

Key Numbers

Parameter	Value
Forecast horizon	6 months
ADS lookback	30 days (rolling from latest data date)
Min available days	5 days (for reliable DS inclusion)
Central estimate	mean_all (average ADS incl. zero-ADS DSes)
Hi bound	P90 per SKU
Lo bound	50% of central
Days per month	30 (fixed)
Fallback (no SKU data)	Global median of mean_all across all SKUs
Excluded SKUs	TCB007, TCB010
Manual-only channel	D2C (channel_id = 10)

Generated by The Cradle Box demand planning system · tcb/forecasting.py